Aurora Files

索引与搜索核心技术说明

（算法·伪代码·性能数据）

skylinezhanghan-wq ·个人非商用项目

2026-11-07

对应发布：v0.1.5 ·基准卷 E:\（约 1.10×10^6 条目）

摘要

本文档面向 Aurora Files（Windows x64，Electron + React）的**卷级索引**与**可取消深搜**子系统，给出架构分解、核心算法的详细伪代码，以及同机实测性能表。重点覆盖：（1）UI/Perf 建索双轨；（2）WalkEngine 引擎链 $mft \to rust \to js$；（3）并行 Dirent BFS 与 IndexDebt 错峰清偿；（4）常驻进程 `aurora-indexd` 的父链紧凑存储、查询与空间债规范化；（5）hydrate-first 二次打开与 USN/脏目录增量；（6）搜索管线（内存/indexd/FTS5/walk 回退）与前端排序。性能数据摘自仓库 `docs/perf-ledger.md`、`docs/indexd-resident.md`、`docs/bakeoff-engine.md`（日期 2026-11-06/07）。

目录

1 问题定义与目标 **3**

1.1 产品约束 . . . 3

1.2 非目标 . . . 3

1.3 符号 . . . 3

2 总体架构 **3**

3 建索双轨：UI 与 Perf **4**

3.1 动机 . . . 4

3.2 参数对照 . . . 4

3.3 解析顺序 . . . 4

4 建索主流程 `ensureIndexed` **6**

5 WalkEngine 与三种枚举引擎 **6**

5.1 优先级 . . . 6

5.2 Bakeoff 同机结果（E:\，约 1.10 M） . . . 6

6 并行 Dirent BFS（JS 引擎） **6**

6.1 设计要点 . . . 6

7 IndexDebt：记债与错峰清偿 **7**
7.1 优先级（数字越小越先） . . . 7
7.2 谷期涓流 FTS（Perf 轨） . . . 9

8 aurora-indexd：父链存储与查询 **9**
8.1 存储模型（v2） . . . 9
8.2 路径 Upsert . . . 9
8.3 MFT 节点 Upsert（无全路径） . . . 10
8.4 查询 . . . 10
8.5 空间债 `normalize_slice` . . . 10

9 搜索管线 **11**
9.1 端到端 . . . 11
9.2 FTS5 侧车 . . . 11
9.3 前端排序 `scoreMatch` . . . 12

10 增量：USN 与 softRefresh **12**

11 性能数据汇总 **12**
11.1 JS lean 栈演进（Node 内 meta） . . . 12
11.2 indexd 常驻演进 . . . 13
11.3 Hydrate-first . . . 13
11.4 冷启动（应用壳，非索引） . . . 13
11.5 UI 虚拟化（附） . . . 13

12 复杂度与正确性要点 **14**

13 复现实验 **14**

14 结论 **14**

1 问题定义与目标

1.1 产品约束

Aurora Files 需要在 Windows 上对**真实磁盘**（非演示沙盒）提供：

- 四视图浏览时界面可交互（建索不得饿死主进程）；
- 卷级名称/路径检索，热查询 ideally $\leq 40\,\text{ms}$；
- 二次打开接近「秒级可搜」（hydrate-first，而非每次冷扫百万）；
- 可选特权路径下利用 NTFS MFT/USN 加速枚举与增量。

1.2 非目标

不在本文档展开：UI 虚拟化、预览/ffmpeg、安装签名。不承诺「首次冷扫秒完百万」——产品口径将**秒级**定义为二次打开的 hydrate + 增量追赶。

1.3 符号

符号	含义
V	卷根，如 `E:\`
N	索引条目数（约 1.1×10^6）
T1	首次可搜时刻（searchable）
T2	T1 之后债清偿（如 FTS）收尾段
RSS	常驻集大小（进程内存）
FRN	NTFS File Reference Number

2 总体架构

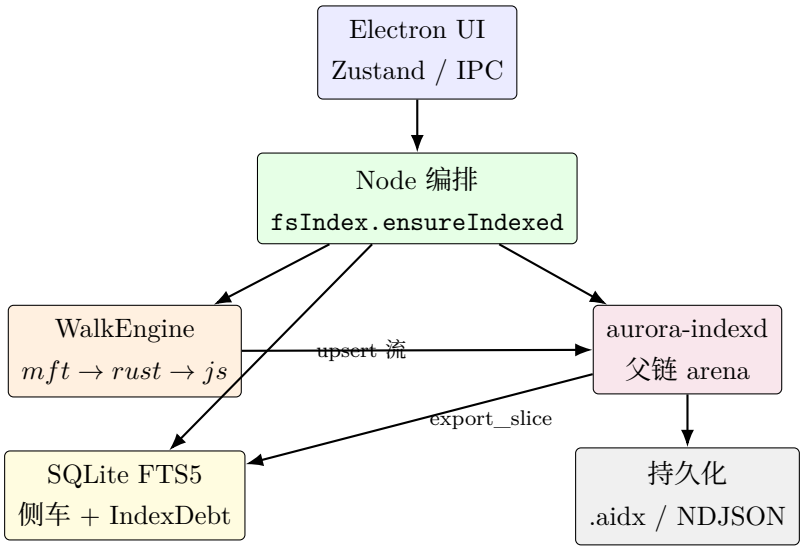


图 1: 索引栈：UI → Node 编排 → WalkEngine 流式 upsert → indexd；FTS 为侧车债。

权威实现入口：

- 编排：`electron/index/fsIndex.mjs`
- 遍历：`electron/index/fastWalk.mjs`、WalkEngine、`native/`
- 查询：`electron/index/searchFiles.mjs`、indexd `query`
- 债：`electron/index/indexDebt.mjs`
- 轨：`electron/index/indexTrack.mjs`

当 `AURORA_INDEX_BACKEND=native`（或缺省且卷根 + 二进制存在）时，N 百万级条目**不**驻留 V8 `meta`；Node 仅编排。JS 后端则 `VolumeIndex.meta` 持全量。

3 建索双轨：UI 与 Perf

3.1 动机

高并发 walk + 同步还 FTS 债会饿死 Electron 主进程，导致进夹/拖动卡顿；压测又需要另一套高吞吐旋钮。二者**完全分轨**，默认互不掺和。

3.2 参数对照

参数	UI（默认）	Perf
目标	建索时界面可操作	冲吞吐 / bakeoff
`maxPerf`	false	true
并发	$\approx \min(12, \text{cpus})$	可达 32–128
`yieldEvery`	512	8192
walk 中 FTS 涓流	**关**	**开**
债清偿节奏	疏（~ 40 ms）	密

3.3 解析顺序

1. `opts.maxPerf === true` ⇒ perf
2. `opts.track` / `opts.indexTrack`
3. 环境变量 `AURORA_INDEX_TRACK`
4. 持久化 prefs `indexTrack`
5. 缺省 **ui**

桌面 IPC `fs:ensureIndex` 默认塞 `track:'ui'`；体积压测应显式 `track:'perf'`。

算法 4.1: ensureIndexed($V, opts$) —可搜优先

输入: 卷根 V，选项（rebuild / hydrateFirst / track / lean …）

输出: VolumeIndex 状态（ready / source / debt）

$opts \leftarrow$ applyIndexTrack($opts$);
$idx \leftarrow$ getIndex(V);
if idx.ready $\wedge \neg rebuild \wedge \neg refresh$ **then**
    attachLiveFollowers(V);
    return idx;
if $refresh \wedge idx$.ready **then**
    return softRefresh(V);
idx.building $\leftarrow true$;
▷ 尝试盘缓存 `hydrate`
if $\neg rebuild \wedge$ wantNative $\wedge$ diskCacheExists **then**
    hydrateNativeFromDisk(idx) ▷ `.aidx / NDJSON`
    ;
    $hydrated \leftarrow true$;
else if $\neg rebuild \wedge$ JS disk entries **then**
    idx.hydrate($entries$);
    $hydrated \leftarrow true$;
$decision \leftarrow$ usnNeedsRebuild($idx.usn$, liveUsn);
if $rebuild \vee decision.rebuild$ **then**
    idx.clear;
    $hydrated \leftarrow false$;
    $mode \leftarrow$ full;
else if $hydrated \wedge$ hydrateFirst **then**
    idx.source $\leftarrow$ `disk-hydrate-first`;
    idx.building $\leftarrow false$;
    异步 softRefresh(V);
    ▷ 不挡可搜
    调度 space 债切片;
    return idx;
▷ 冷扫 / `mtime-delta`
WalkEngine.stream(V) $\xrightarrow{\text{upsert}}$ idx;
记债：fts / frn / persist / space（lean 时）;
idx.ready $\leftarrow true$ ▷ `T1`
;
错峰 paySlice 直至债清或后台;
persist（可异步）;
return idx;

4 建索主流程 `ensureIndexed`

关键产品决策：二次打开走 hydrate-first——T1 可为数十毫秒级「盘缓存可搜」，增量与债后台完成。

5 WalkEngine 与三种枚举引擎

5.1 优先级

$$\text{priority} = \begin{cases} [mft, rust, js] & \text{Admin + SeBackup 且 bakeoff adopt} \\ [rust, js] & \text{无特权或缺省} \end{cases}$$

静默降级：`mft_no_priv` → rust → js，不崩溃。

5.2 Bakeoff 同机结果（$E{:}\backslash$，约 1.10 M）

非特权

引擎	模式	条目	ms	条目/s
JS	store	1.098 M	8342	1.32×10^5
Rust	store	1.099 M	5395	2.04×10^5
C++ MFT	—	—	—	`skipped_no_priv`

Rust store RSS ≈ 15 MB，JS store ≈ 311 MB（约 0.05× 内存、1.56× 吞吐）⇒ **rust adopt**。

特权（Admin）

引擎	模式	条目	ms	条目/s
C++ MFT	store	1.098 M	5602	1.96×10^5
Rust	store	1.116 M	14051	7.94×10^4
JS	store	1.100 M	20146	5.46×10^4

MFT 经 FRN 48-bit 掩码、根 FRN= 5、父路径 memo 后 **adopt**；e2e 上 MFT→indexd 直通后 T1 曾优于纯 rust 路径（见第 11 节）。

6 并行 Dirent BFS（JS 引擎）

实现：`parallelDirentWalk`（`fastWalk.mjs`）。

6.1 设计要点

- 并行 BFS；队列用 **头指针** $qHead$ 避免 `Array.shift` 的 $O(n)$；
- Dirent 优先，文件省略 `kind`（缺省 =file）压对象字段；

算法 5.1: WalkEngine.SelectAndStream(V)

输入: 卷 V，环境 `AURORA_WALK_ENGINE`

输出: 条目流 $\{(\text{path}, \text{name}, \text{kind}, \text{frn?})\}$

```
P ← 解析优先级列表;
for e ∈ P do
    if e = mft then
        启动 aurora-mft-enum（FSCTL_ENUM_USN_DATA）;
        if 无特权或失败 then
            continue;
        流式 NDJSON / stream-nodes（可仅 FRN+name+parent）;
        return;
    if e = rust then
        启动 aurora-walk-bench 子进程，Dirent 流式 upsert;
        if 成功 then
            return;
    if e = js then
        parallelDirentWalk(V);
        return;
```

- `seenDirs` 防环；可选跟随 reparse + `realpath` 去重；
- UI/Perf 控制 *concurrency* 与 *yieldEvery*。

历史优化：同栈从「顺序 lstat + 同步 FTS」到「并行 Dirent + 批量 ingest + 延后二级索引」后，E:\ 约从 38k/300s（~ 128/s）到 1.10M/42s（$\sim 2.6 \times 10^4$/s）。

7 IndexDebt：记债与错峰清偿

热路径只做「可搜最小集」，重活记债后按时间片偿还。

7.1 优先级（数字越小越先）

优先级	种类	含义
0	`fts`	SQLite FTS5 侧车写入
1	`frn`	FRN 懒补（服务 USN）
2	`secondary/persist`	二级结构 / 落盘
3	`space`	indexd 空间规范化
4	`stat_meta`	元数据补齐

`space` **不**阻挡可搜与落盘。

算法 6.1: parallelDirentWalk($root, opts$)

输入: 根路径，$maxScan$，$maxDepth$，并发度 C，跳过谓词 $skip$

输出: 经 $onEntry$ 回调的条目；$scanned$

$queue \leftarrow [(root, 0)]$;
$qHead \leftarrow 0$;
$seen \leftarrow \{root.\text{lower}\}$;
$scanned \leftarrow 0$;
while $\neg stop()$ **do**
  while $inFlight < C \wedge queueLen() > 0$ **do**
    $(dir, depth) \leftarrow queue[qHead++]$;
    异步：$ents \leftarrow \text{readdir}(dir, \{\text{withFileTypes}\})$;
    for $ent \in ents$ **do**
      if $skip(ent.name) \vee \text{symlink}$ **then**
        continue;
      $full \leftarrow \text{joinLogical}(dir, name)$;
      $scanned \leftarrow scanned + 1$;
      $onEntry(\text{entryFromDirent}(full, name, ent))$;
      if $ent.\text{isDirectory}() \wedge depth < maxDepth$ **then**
        if $full.\text{lower} \notin seen$ **then**
          $seen.\text{add}(full.\text{lower})$;
          $queue.\text{push}((full, depth + 1))$;
      if $scanned \bmod yieldEvery = 0$ **then**
        await setImmediate;
    $inFlight \leftarrow inFlight - 1$;
    唤醒等待者;
  if $inFlight = 0 \wedge queueLen() = 0$ **then**
    break;
  await 有完成事件;

算法 7.1: IndexDebt.paySlice($budget$)

输入: 时间预算 ms，操作预算 ops

输出: $(paid, kind)$

if $paused \vee paying \vee payer = \bot$ **then**
  return $(0, \bot)$;
$ordered \leftarrow$ 已欠债种类按 PRIORITY 升序;
if $ordered = \emptyset$ **then**
  return $(0, \bot)$;
$kind \leftarrow ordered[0]$;
$n \leftarrow payer(kind, \{ms, ops\})$;
累计 $paid$；若该类完成则 `clear`;
return $(n, kind)$;

7.2 谷期涓流 FTS（Perf 轨）

冷扫速率非匀速：峰可达 $\sim 10^5$/s，谷可降至 $\sim 2 \times 10^4$/s（差约 5×）。Perf 轨在瞬时速率低于阈值（如 $< 5 \times 10^4$/s）时小批量 `INSERT`，缩短 T2 且尽量不拉长 T1。实测修复空 meta 误清债后，T1 可消化约 13% FTS（$\sim$ 14 万条）而 T1 几乎不变（$\sim$ 24.7 s vs $\sim$ 24.5 s）。

8 aurora-indexd：父链存储与查询

8.1 存储模型（v2）

- **Arena**：唯一名称分量字节池;
- **Entry**：$\{parent, name_off, flags, frn, path_hash\}$;
- 路径按需 `build_path` 沿父链拼回;
- 快照 `.aidx`（magic `AIDX` v2）；增量 `.aidx.d`;
- FRN：稠密表 + overflow；MFT 路径用 `upsert_node` 延迟父链接（`barrier`/`seal`）。

8.2 路径 Upsert

算法 8.1: indexd.upsert($path, name, is_dir, frn$)

$path_{lc} \leftarrow$ lower($path$)；$comps \leftarrow$ split($path_{lc}$);
校正叶名与 $name$ 一致;
ensure_components($comps, is_dir, frn$)：逐级 intern 名、分配/复用 entry、挂 parent、更新 path 槽与 FRN;
标记 $name_index_dirty$;

8.3 MFT 节点 Upsert（无全路径）

算法 8.2: indexd.upsert_node($name, frn, parent_frn, is_dir$)

输入: 名称与 FRN 三元组

if $frn = 0 \vee name = \varepsilon$ **then**
  return;
冷启动首次 persist 前：*push_bytes*（跳过 name_intern 哈希，省 RSS）;
if $\exists id =$ frn_get(frn) **then**
  更新 name/flags；登记 $pending_parent[id] \leftarrow parent_frn$;
else
  分配新 entry；同上挂 pending;
$path_index_ready \leftarrow false$;
脏名索引;

`seal/barrier` 时根据 FRN 解析 parent 指针，避免 walk 时物化百万全路径字符串。

8.4 查询

多词 AND：主词走名称索引（前缀或 contains），其余在 name + path 上过滤；无需 FTS。短词（≤ 4）或多词强制 contains 扫描，避免错误前缀切断。

算法 8.3: indexd.query($q, limit, mode$)

输出: 至多 *limit* 条 EntryView

seal();
$tokens \leftarrow$ split(lower(q));
$primary \leftarrow tokens[0]$； $rest \leftarrow tokens[1:]$;
if $mode =$ `contains` $\vee\ |tokens| > 1 \vee |primary| \leq 4$ **then**
  for $id \in name_index$（不整表克隆）**do**
    if live $\wedge\ primary \subseteq name(id) \wedge$ matches_rest($view, rest$) **then**
      输出；满则返回;
else
  $start \leftarrow$ partition_point($name_index$, name $<$ $primary$);
  for $id \in name_index[start:]$ **do**
    if ¬name.starts_with($primary$) **then**
      break;
    过滤 $rest$；收集至 $limit$;

8.5 空间债 `normalize_slice`

相位机：

1. `path`：批填 *path_hash* + open-addressing `path_slots`；

2. `shrink`：收缩 intern/FRN 等；

3. `arena`：至多一次 `compact_arena_entries`，然后可能回到 path；

4. `done`：复位。

`.aidx.d` 超过阈值（默认 8 MB）时 coalesce 为全量 `persist_bin`。

9 搜索管线

9.1 端到端

算法 9.1: searchFiles($root, query, opts$)

$q \leftarrow \text{lower}(\text{trim}(query))$;
if $q = \varepsilon$ **then**
  return 空;
$jobId \leftarrow ++ globalSearchJob$;
▷ 取消令牌
异步 ensureIndexed（不阻塞过久）;
$indexed \leftarrow \text{queryIndex}(root, q)$;
if 已取消 **then**
  return cancelled;
if *indexed* 有命中或 *source* 以 ***index*** 开头 **then**
  return *indexed*;
▷ 回退：浅层可取消 `walk`
受限 $maxDepth \le 6$, $maxScan \le 5 \times 10^4$;
DFS/BFS 扫描；名包含 q 则收集;
return $\{items, source = \texttt{walk-fallback}\}$;

`queryIndex`：若未 ready 且扫描极少，先 `warmShallow`；否则调用 *idx.query*（JS meta / FTS / indexd）。

9.2 FTS5 侧车

表:`files(path UNINDEXED, name)`,`unicode61` 分词;WAL;流式 `beginRebuild/insertFast/endRebuild` 避免全量数组尖峰。查询:

```
SELECT path, name FROM files WHERE files MATCH ? LIMIT ?
```

Native 默认可跳过 FTS（indexd 多词足够）；`AURORA_INDEXD_FTS=1` 时启用。

9.3 前端排序 `scoreMatch`

$$s = \begin{cases} 1000 & name = q \\ 500 & name.\text{startsWith}(q) \\ 200 & q \subseteq name \\ 80 & q \subseteq path \\ 0 & \text{否则（丢弃）} \end{cases} + [5 \text{ if file}] + \max(0, 40 - |name|)$$

再按分数降序、`zh-CN` 名比较截断。支持 `tag:` / `type:` 前缀解析。

10 增量：USN 与 softRefresh

- 冷扫 lean 不 `stat` ⇒ FRN 稀疏；用 `debt.frn` 懒补，禁止为 FRN 全量阻塞 T1。
- `softRefresh` = ReadUsn(FRN) + 脏目录重扫；无特权时脏目录路径仍正确。
- 小树基准（`test:perf-diff`，$K = 100$）：index ~ 129 ms，refresh ~ 839 ms（脏路径）；有 USN 权限后目标 p95< 50 ms。

11 性能数据汇总

除非另注，体积 $\approx E$:\、$N \approx 1.10 \times 10^6$，日期 2026-11-06/07。

11.1 JS lean 栈演进（Node 内 meta）

阶段	T1	峰 RSS	热搜
早期并行 + 同步 FTS 思路	~ 42 s 量级	~ 2.8 GB 封口	~ 32 ms
lean + IndexDebt	~ 24 s	~ 755 MB	~ 20 ms
maxPerf 极限	10.2 s	—	21 ms
空间极限（弃 byName 等）	9.1 s / 494 MB	搜后 ~ 370 MB	28 ms

11.2 indexd 常驻演进

阶段	T1	Node RSS	indexd RSS
First native	12 s	252	425
去重键 / 直通 NDJSON	8.9 s	113	170
父链 + `.aidx`	9.4 s	49	92
MFT FRN 直通	7.3 s	50	126
dense FRN + 快 aidx	∼ 11 s	∼ 107	62
延迟 path_hash	4.4 s	105	62
space + normalize	4.3 s	—	96（path on）
path_slots + 无 MFT intern	**6.2** s	**82**	**69**

最新门槛（elevated MFT，run `indexd-2026-11-07T02-46-48`）：

指标	实测	门槛
T1 searchable	6.2 s	—
space_cleared	2.6 s（异步）	不计入 T1
persist	101 ms	—
Node RSS	82 MB	≤ 90 / 产品 ≤ 150
indexd RSS（path ready）	69 MB	≤ 90 / 产品 ≤ 100
冷 / 热搜	4.2 / 0 ms	热 ≤ 40
path_index_ready	true	要求

完整评估复跑（×3）：T1 6.2–7.1 s，Node 77–79，indexd 68–70，冷搜 3–8 ms，热搜 0，bakeoff mft/rust adopt，warn= []。

11.3 Hydrate-first

样例：可搜 ∼ 40 ms，热搜 1 ms，source = `disk-hydrate-first`（`test:perf-hydrate`）。

11.4 冷启动（应用壳，非索引）

asar 291 MB → 5.6 MB 等优化后：median ready 516 ms / interactive 748 ms（弃首跑 AV）；门槛 ready≤ 2500，interactive≤ 4500。首启仍可能受 Defender 影响至数秒。

11.5 UI 虚拟化（附）

S-mem open ∼ 357 ms，paced/fling longtask 0；大目录 DOM ≪ N（list/col/gal/grid 约 28/43/15/42）。

12 复杂度与正确性要点

操作	时间（均摊/期望）	备注
Dirent BFS 枚举	$\Theta(N)$ IO + CPU	受磁盘与杀软谷期主导
path upsert（父链）	$O(\text{depth})$	分量 intern
前缀 query	$O(\log N + K)$	排序名索引 + 扫描前缀段
contains query	$O(N)$ 最坏	短词/多词；早停 K
paySlice	$O(ops)$	受 ms 预算截断
hydrate .aidx	$\Theta(N)$ 顺序读	不经 V8 解析百万行

正确性注意：

- 搜索取消靠单调 $jobId$，旧任务结果丢弃；
- hydrate 代数与 `generation` 防止过期 walk 写回；
- MFT 无特权必须降级，不可空结果冒充完成；
- FTS 涓流不得在 $meta.size = 0$ 时误 clear 债。

13 复现实验

```
# WSL 或 Windows（索引压测在 Windows 主机）
npm ci
npm run native:indexd
npm run native:walk-bench
# 可选特权
npm run native:mft-enum

npm run test:perf-volume          # Perf 轨整卷
npm run test:perf-indexd          # indexd 门槛
npm run test:perf-hydrate
npm run test:perf-bakeoff -- --volume E:\
npm run test:perf-bakeoff:elevated
```

本 PDF 编译（WSL）：

```
cd docs/tex
xelatex -interaction=nonstopmode aurora-index-search.tex
xelatex -interaction=nonstopmode aurora-index-search.tex
```

原始叙事与逐次 runId：`docs/perf-ledger.md`、`docs/indexd-resident.md`、`docs/bakeoff-engine.md`、`docs/index-tracks.md`、`docs/research-volume-scan-diff.md`。

14 结论

Aurora Files 的索引/搜索核心可概括为四句话：

1. **可搜与完备分离**——T1 最小可搜 + IndexDebt 错峰；
2. **引擎可替换**——WalkEngine $mft \to rust \to js$ 静默降级；
3. **权威数据出 V8**——indexd 父链 + `.aidx`，Node 编排；
4. **二次打开 hydrate-first**——秒级体感来自盘缓存，而非承诺首次冷扫秒完。

在个人非商用场景下，同机百万级卷已达到：T1 约 6–7 s（冷扫）或数十毫秒（hydrate），热搜亚毫秒至数毫秒，Node+indexd RSS 合计约 150 MB 量级——足以支撑日常自用文件管理器的搜索体验。

*MIT License ·Aurora Files v0.1.5 ·*本文档随仓库 *`docs/tex/`* 维护。